PYTHON · FLASK · FAISS + BM25 · AZURE AI FOUNDRY · AKS · ISTIO

ONE API Agent

A multi-agent service that answers natural-language questions about applications, cost centres, platforms and incidents — with citations, a confidence score, and a hard-wired willingness to say *I don't know*.

Problem	Estate truth is spread across an internal API, a read-only Postgres store, a published documentation index and ServiceNow. Four access paths, four query dialects, four owning teams.
Approach	Four thin domain agents behind one OpenAI Assistants-compatible protocol, sharing a single grounding layer: hybrid retrieval, confidence gating, citation enforcement and telemetry.
Stack	Python 3.13, Flask 3, Gunicorn, FAISS, BM25Okapi, Azure AI Foundry, PostgreSQL, Docker, AKS, Istio, New Relic.
My role	End-to-end: retrieval architecture, confidence model, agent design, A2A protocol layer, Kubernetes deployment and evaluation harness.

The problem

A large amount of senior engineering time goes to questions that are not engineering at all. *Who owns this application? Which tower does this cost centre roll up to? What Kubernetes version is platform Y running? What changed before last night's P1?*

Each is a thirty-second lookup — **if** you know which system holds the answer and what the field is called. That conditional is the whole problem: the human becomes the integration layer, and context-switching is the tax paid on every question.

Why a generic copilot makes it worse

Putting an off-the-shelf LLM in front of everything fails for a reason that is easy to state and hard to engineer around: **a general-purpose model is optimised to produce a plausible answer, and the enterprise needs a provable one.** A hallucinated application owner does not look wrong — it looks exactly like a correct answer, and it flows silently into a chargeback dispute, an access review or a P1 bridge.

Dimension	Generic LLM copilot	ONE API Agent
Source of answer	Model weights plus whatever it can reach	Only retrieved, citable chunks and live API data
On weak evidence	Answers anyway, confidently	Refuses — *insufficient_evidence*, with no LLM call at all
Citations	Often fabricated or unverifiable	Invented chunk ids stripped; answer discarded if none survive
Confidence	Opaque or absent	Transparent multi-factor blend; every sub-score returned
Auditability	No trace	Every query — grounded or refused — recorded as telemetry

Dimension	Generic LLM copilot	ONE API Agent
Regression safety	Prompt-tuning by hope	Golden evals assert grounding and refusal in CI

> **The design question**
>
> It was never “which model?” It was: what control plane do you build around a model so that groundedness becomes a property of the *system* rather than a property of the prompt?

ARCHITECTURE

One service, four agents, one grounding layer

Flask behind Gunicorn, containerised on AKS and exposed through an Istio VirtualService. Every conversational surface enters through the same door: an OpenAI Assistants-compatible agent-to-agent protocol with Entra ID / MSAL authentication and a discovery card, which is what makes the service pluggable into Copilot Studio without writing a connector.

CHANNELS	Copilot Studio	Azure AI Foundry	Self-hosted chat UI	REST / Swagger
ENTRY · A2A	Assistants-compatible threads & runs	MSAL / Entra ID auth	Istio ingress on AKS	
AGENTS	ONE API apps & infra	Cost Center finance hierarchy	ETS Platforms RAG over docs	Incident ServiceNow + GitHub
GROUNDING	Hybrid BM25 + FAISS	Confidence gate	Citation enforcement	Query cache + telemetry
SOURCES	ONE API	Insights Postgres (read-only)	Status-page doc index	ServiceNow / GitHub Actions

Agents are thin and specialised; the grounding layer is shared. A new domain inherits confidence scoring, citation enforcement and telemetry on day one — and, just as importantly, cannot quietly opt out of them.

Groundedness as a pipeline, not a prompt

Generation is a **fixed pipeline**. There is no agentic free-for-all in which the model decides whether to search: retrieval and gating are deterministic and happen first, generation is a single tool-less call, and the output is post-processed before a user ever sees it.

1	2	3	4	5	6	7	8
Ingest	**Hybrid retrieve**	**Fuse & rerank**	**Filter**	**Score**	**Gate**	**Generate**	**Enforce**
Per-platform JSON chunks with inline embeddings, cached locally	FAISS cosine + BM25 keyword, side by side	Reciprocal Rank Fusion, then optional LLM rerank	Drop chunks below the per-chunk floor (0.30)	Semantic + lexical + margin + authority × freshness	Below 0.35: refuse, with zero LLM calls	Stuffed context, no tools — cannot fetch or invent	Strip unknown chunk ids; discard if none survive

Outcome	What the caller receives
Grounded answer	Cited text plus a clickable Sources panel — chunk id, heading, platform, commit timestamp and an absolute URL to the live document.
Refusal	*insufficient_evidence*, grounded: false — explainable, because every confidence sub-score is returned alongside it.
Telemetry	Index version, candidate chunk ids, confidence and gate decision, emitted for grounded and refused queries alike.

> **Refusal is a feature**
>
> Stages 4, 6 and 8 are three independent opportunities to stop a bad answer — two of them before a single token is generated. The system is designed to say “I don't know” rather than produce an unverifiable answer.

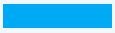
Why hybrid retrieval, not just embeddings

Pure vector search is excellent at paraphrase and terrible at identifiers — ask about a version string or an error code and cosine similarity returns a semantically adjacent paragraph that does not contain the token. Pure keyword search has the mirror failure. So the index is built twice over the same chunks: an in-memory FAISS *IndexFlatIP* over L2-normalised vectors, and a BM25Okapi index. The two rankings merge with Reciprocal Rank Fusion, which combines *ranks* rather than scores and so needs no normalisation between two incomparable scales.

CONFIDENCE

A score you can argue with

Deliberately not a learned scorer — a transparent blend of four normalised signals with environment-tunable weights, every sub-score returned with the answer.

Top semantic cosine		**0.55**	Strength of the best embedding match
BM25 lexical agreement		**0.20**	Does the keyword index agree with the vectors?
Top-vs-second margin		**0.10**	Is the winner clearly ahead of the runner-up?
Authority × freshness		**0.15**	Doc-type authority, decayed on a 180-day half-life over git commit time

Gate	Rule	Effect
Query gate	confidence ≥ 0.35	Below it the agent refuses before any LLM call is made.
Chunk floor	per-chunk ≥ 0.30	Weak evidence never reaches the prompt.
Scope guard	0.35 / 0.45 bounds	An ambiguous platform choice falls back to searching all.

Authority × freshness encodes something a model has no way to know: that a generated versions file is more authoritative than an architecture note, which is more authoritative than a runbook, and that a document last committed eighteen months ago should be trusted less than one committed last week. Because every sub-score is

returned, a refusal is explainable — you can see whether nothing matched semantically, or the top two candidates were indistinguishable, or the only supporting document was stale.

ENGINEERING

The parts that are not about AI at all

Decision	Why
Stateless, context-carrying follow-ups	Chat history is never injected into the grounded prompt — stacking a fresh grounded instruction on a prior one reliably pushes models into refusing. Recent turns are folded into a *contextualised retrieval query* instead, which is also used as the grounded question. One deterministic carrier of conversation context.
Enumeration-aware retrieval	A single global top-k starves “list all X across every platform” queries. Rather than a brittle intent classifier, the strategy is derived from *scope cardinality*: one platform means focused search; two or more means per-platform retrieval, unioned and re-ranked.
An incident agent with no database	GitHub Actions is the job store — each run is identified by its run name and returns a result artifact. Thread ownership lives as a hashed owner email in conversation metadata. No schema, no migrations, no state to drift.
Idempotency, guarded twice	A non-blocking per-ticket lock inside the pod *and* a per-ticket workflow concurrency group upstream, because the two failure modes differ: concurrent requests to one pod, and concurrent requests across pods.
Memory as a capacity decision	Each Gunicorn worker holds its own in-memory FAISS + BM25 + embeddings copy, so per-pod memory scales with worker count — which is why pod limits are sized explicitly rather than inherited. Database access is read-only by credential, not by convention.

Caching, and being honest about it

A cache hit returns the stored answer and its sources, skipping retrieval *and* generation entirely, keyed on query, index version and platform scope so a re-published index invalidates naturally. Refusals are never cached, because the missing evidence may arrive tomorrow.

The caveat is published rather than hidden: the cache is **pod-local SQLite** — shared across a pod’s workers via WAL, but not across replicas and not durable across restarts. A deliberate stage-one trade-off, and the first item on the roadmap. Documenting the limitation is part of the same discipline as refusing an ungrounded answer.

OUTCOME

What it delivers

Audience	Value
Engineers	Plain-language access without learning four query dialects, with follow-up questions that retain their context.
Finance & risk	Every cost-centre or approver answer arrives with a source link and a confidence score that can be defended in a review.
Incident response	Ticket analysis dispatched and tracked without standing up another database, with duplicate-run protection on the bridge.
The platform	Grounding, confidence, citation and telemetry are shared services — the next agent inherits them on day one.

Roadmap

- **Durable shared cache** across replicas, replacing the pod-local SQLite store.
- **Close the rolling-deploy window** in which two pods can create one duplicate queued run.
- **Expand golden evals** to the cost-centre and ONE API agents, not just platforms.
- **Publish the grounding layer** as a reusable internal library so confidence thresholds become a governed, per-domain risk setting.

> **The transferable lesson**
>
> Fluency is cheap and getting cheaper. Traceability is the scarce thing — and it is an engineering property, not a model capability. Decide your refusal policy before your model: “what happens when we don’t know?” is an architecture question, and answering it late means retrofitting gates around a system never designed to stop.